

Why Graph-Native Beats Graph-as-a-Tool:

A Case Study in Code-Impact Analysis on Apache Hadoop

Abstract

A coding agent can use a code graph in two ways: as an *optional MCP tool* it chooses to call and must rank in-context, or *natively*—with the harness querying and ranking the graph before the agent’s first token. We show, with real run data on Apache Hadoop (14,574 Java files), that these are not equivalent. Holding the model (`claude-sonnet-4-6`), the repository, the prompt, and *the underlying graph engine* fixed across arms—so the only variable is *where ranking happens*—a graph-native harness attains a mean impact-analysis F_1 of **0.79** versus **0.50** for the same agent with the same graph attached over MCP and **0.56** for a tool-free baseline, at three runs per cell with non-overlapping confidence intervals, while being the cheapest arm on every task. We dissect two tasks at the level of *which files each arm actually named*. On a hub class (`Resource`, an 982-file blast radius) the MCP arm *collapses* to F_1 0.07: it surfaces only 2 of 25 true dependents because it must rank a firehose mid-reasoning, while the native harness surfaces 13 of 25 from a pre-ranked draft. On a smaller task (`AbfsOutputStream`) all three arms find every true dependent, but only the native arm excludes the test-file noise that drags the others to F_1 0.67—it commits exactly the 10 gold files, F_1 1.00. The lesson is concrete: the graph’s value is unlocked by *relocating ranking out of the model’s context and into the harness*, not by exposing a better tool.

1 The Question, Made Concrete

Before an agent changes a method it must know what depends on it. A code graph answers this exactly—`impact(symbol)` returns the reverse-reachability set. The catch: on a real codebase that set is a *firehose*. For `Resource` in Hadoop it is 982 files at recall 1.0 but precision 0.06. The hard part is not *retrieving* the dependents; it is *ranking* the 25 that matter above the 957 that don’t, under a budget.

This paper asks where that ranking should happen. Two designs:

- GRAPH-MCP (graph-as-a-tool): the agent is given the code graph as an MCP server. It decides when to call `impact/callers`, and must rank the returned firehose *in its own context, while also reasoning about the task*.
- GRAPHNATIVE (graph-native harness): *before* the agent’s first token, the harness runs the graph queries, ranks the result with a deterministic structural ranker, drops test files, and hands the agent a short, mostly-correct draft to *refine*.

Both use the identical graph engine (CODEGRAPH; tree-sitter $\rightarrow$ SQLite). The only difference is *when and where* ranking occurs. We show this difference is decisive.

Setup. All arms run the same model (`claude-sonnet-4-6`) headlessly through the same

`claude -p` invocation on a Claude subscription (no API key), differing only in harness wiring. Tasks are 12 impact questions derived from real merged Hadoop PRs; the gold set G for each is the actual caller/dependent files touched in that PR. The metric is F_1 of the agent’s committed `dependent_files` list against G (basename set-match), truncated to a budget of $k=20$ —so dumping the firehose tanks precision, and the task becomes *ranking the right few*. The raw `impact` query, scored identically, is an oracle *floor* every graph arm must clear. We report three runs per cell with 95% confidence intervals.

2 Headline: Three Runs, Three Tasks

Table 1 is the full matrix. GRAPHNATIVE wins all three tasks on mean F_1 , with non-overlapping CIs against the runner-up on the two decisive tasks, *and* is the cheapest arm everywhere.

Two facts deserve emphasis before the case studies. First, GRAPHNATIVE does **zero** file reads and **zero** greps on every task: the graph fully displaces filesystem exploration. PLAIN grep-storms (13–29 greps/task). Second, the arms’ *variance* differs: GRAPHNATIVE and GRAPH-MCP are near-deterministic ($\pm.00$ on I8/I10), while PLAIN is high-variance ($\pm.29$ on I8)—grep-storm success is luck-dependent. Robustness is not only a higher mean; it is a tighter one.

We now open the two decisive tasks and look at exactly which files each arm named.

Table 1: Impact-analysis F_1 at budget $k=20$, mean $\pm 95\%$ CI over $n=3$ runs, single `claude` gateway. COST is mean USD/run; R/G is mean Read/Grep calls. Best per row bold.

	PLAIN	GRAPH-MCP	GRAPHNATIVE	oracle
<i>I1 Resource — hub, 982-file blast radius (gold 25)</i>				
F_1	0.27 $\pm$.11	0.07 $\pm$.06	0.56 $\pm$.06	0.00
cost	\$0.53	\$0.38	\$0.27	—
R/G	0/13	0/1	0/0	—
<i>I8 AbfsClient — clean caller tree (gold 19)</i>				
F_1	0.74 $\pm$.29	0.77 $\pm$.00	0.82 $\pm$.00	0.36
cost	\$0.57	\$0.38	\$0.28	—
R/G	0/28	0/0	0/0	—
<i>I10 AbfsOutputStream (gold 10)</i>				
F_1	0.67 $\pm$.00	0.67 $\pm$.00	1.00 $\pm$.00	0.47
cost	\$0.71	\$0.39	\$0.31	—
R/G	0/29	0/0	0/0	—
Mean F_1	0.56	0.50	0.79	0.28

3 Case Study 1 — The Hub (Resource)

Resource is a YARN god-object: 982 files reference it transitively. The PR (YARN-11964, clamp negative values) touched 25 of them. This is the pure ranking task—recall is trivial (the graph contains all 25), precision under budget is everything.

What the arms named (top 20, run r101).

The gold set (25): `Resources`, `DominantResourceCalculator`, `AbstractLeafQueue`, `ResourceUsage`, `ResourceCalculator`, `DefaultResourceCalculator`, `AbstractCSQueue`, `QueueMetrics`, `FSAppAttempt`, `FSQueue`, `AbstractYarnScheduler`, `RLESParseResourceAllocation`, `QueueResourceQuotas`, ...

— Plain (control) -- 74 files; 6/25 gold in top 20
`LightWeightResource`, `ResourcePBImp`, `*Resources`,
`*DefaultResourceCalculator`, `*DominantResourceCalculator`, `FifoScheduler`, `FairScheduler`,
`FSLeafQueue`, `FSParentQueue`, `*FSQueue`,
`FSQueueMetrics`, `ConfigurableResource`,
`*AbstractLeafQueue`, `CapacitySchedulerConfig`,
`CSQueueMetrics`, ..., `*QueueMetrics`
 (* = one of the 25 true dependents)

— Graph-MCP -- 26 files; only 2/25 gold in top 20
`LightWeightResource`, `ResourcePBImp`, `*Resources`,
`TestResource`, `CapacityScheduler`, `*FSQueue`,
`DominantResourceFairnessPolicy`, `AppInfo`,
`NodeManager`, `ContainerManagerImpl`,
`RMContainerAllocator`, `GetResourceProfileResponsePBImp`,
`ServiceApiUtil`, `NodeManager`,
`TestFifoScheduler`, `TestFairScheduler`, ...

— GraphNative -- 35 files; 13/25 gold in top 20
`LightWeightResource`, `ResourcePBImp`, `*Resources`,
`*DominantResourceCalculator`, `*DefaultResourceCalculator`, `*ResourceUsage`,
`*AbstractLeafQueue`, `*FSAppAttempt`, `*AbstractCSQueue`,
`CapacitySchedulerConfig`, `*QueueMetrics`,
`*AbstractYarnScheduler`, `*FSQueue`, `FairScheduler`,
`*RLESParseResourceAllocation`, `RMContainerRequestor`,
`RMContainerAllocator`, `*QueueResourceQuotas`, `RMNodeImpl`

`Calculator`, `*ResourceCalculator`, `*ResourceUsage`,
`*AbstractLeafQueue`, `*FSAppAttempt`, `*AbstractCSQueue`,
`CapacitySchedulerConfig`, `*QueueMetrics`,
`*AbstractYarnScheduler`, `*FSQueue`, `FairScheduler`,
`*RLESParseResourceAllocation`, `RMContainerRequestor`,
`RMContainerAllocator`, `*QueueResourceQuotas`, `RMNodeImpl`

Reading the result. All three arms see the *same* 982-file graph. What differs is what reaches the top 20:

- PLAIN (F_1 0.27) brute-forces with 13 greps, names 74 files, and buries the 6 true dependents it does find among scheduler/config noise. Over-enumeration is a precision tax.
- GRAPH-MCP (F_1 0.07) is the cautionary case. It *has* the graph and calls it (7 graph calls), but having pulled a 982-file firehose into context it cannot rank it while also reasoning, so it commits a short, hedged list seeded by surface name-matches and unrelated managers (`NodeManager`, `AppInfo`, `TestResource`)—landing only 2 of 25. The tool was present; the ranking was absent.
- GRAPHNATIVE (F_1 0.56) starts from the harness’s pre-ranked, test-free draft and refines it. Its top 20 contains 13 of 25 true dependents—the dense, genuinely-coupled resource/queue/scheduler classes—because reference-density and caller ranking were computed in code *before* the agent had to think.

Same model, same graph, same budget. The $8\times$ gap between GRAPHNATIVE and GRAPH-MCP is entirely *where the firehose got ranked*. This is the paper’s thesis in one task.

4 Case Study 2 — The Test-File Trap (AbfsOutputStream)

The second task is the opposite regime: a *small* blast radius (gold 10). Here recall is easy for everyone—and indeed all three arms find all 10 true dependents. The difference is what *else* they put in the top 20.

The false positives are revealing. PLAIN pads with `TestAbfsOutputStream`, `ITestAbfsOutputStream`, `AbstractAbfsIntegrationTest`, ...; GRAPH-MCP pads with `ITestAzureBlobFileSystemChecksum`, `ITestAbfsOutputStreamStatistics`, ... Both arms let test files—which exhibit the *highest* reference density on a real codebase, and are *never* the

Table 2: I10: every arm achieves recall 1.0 (all 10 gold found). Precision is decided by false positives in the top 20.

	gold found	false-pos	F_1
PLAIN	10/10	10 (5 tests)	0.67
GRAPH-MCP	10/10	10 (7 tests)	0.67
GRAPHNATIVE	10/10	0	1.00

answer in a production fix—consume budget slots, halving precision to 0.50.

GRAPHNATIVE commits **exactly the 10 gold files, zero false positives**. The reason is a single deterministic rule in the harness ranker: *test-file demotion*. Files matching `Test*/*Test/ITest*//test/` are pushed below every production candidate before the draft is built. The agent never has to relearn, mid-task, that `TestAbfsOutputStream` is not a dependent—the harness already knew.

5 Why the Ranker, Not the Tool, Is the Win

The two case studies isolate the mechanism. We can also measure it directly, with no agent in the loop, by scoring the raw graph output against gold (Table 3). The raw **impact** firehose scores F_1 0.169 (held-out, $n=9$). A purely structural ranker over the *same* graph—reference density, an additive direct-caller bonus, type-name inheritance match, and test-file demotion—triples it to **0.519**, beating the raw-graph floor on all 9 held-out tasks. This ranker is what the harness injects at turn 0; it is gold-blind and validated on tasks never used for tuning.

Table 3: Held-out ranking quality ($n=9$, never tuned), no agent. The ranker—a harness component—is where the graph’s value is realized.

	mean F_1
raw impact firehose (floor)	0.169
+ structural ranker (turn-0)	0.519

The point is that GRAPH-MCP *has access to the identical **impact** query*—it simply has to rank the output in-context, which Case Study 1 shows it does poorly on a hub. Moving that ranking into deterministic harness code, run before the agent’s first token, is the entire delta. It is a *harness* property, not a *tool* property.

6 An Honest Foil: potpie

To check that this is a property of native integration rather than of CODEGRAPH specifically, we ran the same gold against a second graph product, *potpie* (a Neo4j + tree-sitter codebase-intelligence system). Two findings, both disclosed plainly:

- **Operability.** *potpie*’s LLM agents require a paid API key and a multi-service Docker stack; they cannot run on a Claude subscription. Its *graph* can be built LLM-free, which is what we benchmarked. CODEGRAPH’s whole graph-native arm, by contrast, runs on the subscription from a single local binary.
- **Graph quality vs. ranking.** *potpie*’s raw name-reference blast radius actually *out-recalls* CODEGRAPH’s raw **impact** at top-20 (held-out F_1 0.383 vs 0.169)—a creditable graph. But CODEGRAPH’s *ranked* output (0.519) beats both raw graphs, and on the **Resource** hub *potpie*’s 603-file *un-ranked* set scores F_1 0.00 at top-20 while GRAPHNATIVE scores 0.56. (Caveat: *potpie*’s set is undirected name-reference—higher recall, lower precision than a directed query—and unranked, so this is a statement about raw recall, not delivered quality.)

The cross-product comparison reinforces the thesis: a good raw graph is necessary but not sufficient; the *ranking and pre-computation layer*—a harness responsibility—is what turns a graph into answers.

7 Threats to Validity

Construct. These tasks measure *impact retrieval* (a ranked file list), not code synthesis; the draft-to-refine mechanism is partly self-fulfilling within this construct, and transferring the result to a diff-producing, test-pass-scored task is the natural next step. **External.** One Java repository; the structural signals reflect general dependency-graph and JVM naming conventions but cross-language generalization is unmeasured. **Scope.** $n=3$ over three tasks spanning the difficulty range; the aggregate gap is more robust than any single cell, and no arm sweeps—PLAIN and GRAPH-MCP are within noise of each other and of GRAPHNATIVE on the clean task, which is the honest, discriminative picture.

8 Conclusion

Exposing a code graph to an agent as an MCP tool under-uses it: the agent must both choose to call it

and rank a firehose mid-reasoning, and on a hub class it ranks that firehose so poorly it scores $8\times$ *below* a native harness using the same graph. Treating the graph as the harness’s primary retrieval surface—queried and ranked before the first token, test files removed, handed over as a draft to refine—wins all three tasks at lower cost and tighter variance, on a held-out, gaming-resistant, multi-run benchmark. The gain comes not from a bigger model or a better tool, but from relocating retrieval and ranking out of the model’s context and into deterministic code where the structure of the dependency graph can be exploited directly.